

Assignment 3

End-to-End Hugging Face Model Training and Docker Deployment

Akanksha Kapil
M25CSA033

February 26, 2026

1 Introduction

This assignment presents the implementation of a complete end-to-end Machine Learning workflow using the Hugging Face ecosystem and Docker containerization. The objective of the project was to transform an experimental Jupyter notebook into a production-ready, modular pipeline capable of training, evaluating, deploying, and reproducing a transformer-based text classification model.

The workflow covers the full lifecycle of a machine learning system, including model fine-tuning using the Hugging Face Trainer API, evaluation using standard performance metrics, deployment to the Hugging Face Hub, and containerization using Docker to ensure reproducibility and portability across environments.

The classification task focuses on predicting book genres using a fine-tuned transformer architecture.

2 Project Structure

```
project/  
  src/  
    data.py  
    train.py  
    evaluate.py  
  Dockerfile.train  
  Dockerfile.eval  
  requirements.txt
```

This modular structure improves maintainability and reproducibility.

3 Environment Setup Using Docker

3.1 Base Image

```
python:3.10-slim
```

3.2 Installed Dependencies

- transformers
- datasets
- torch
- scikit-learn
- numpy
- pandas
- huggingface-hub

4 Model Selection

Selected model:

prajjwal1/bert-tiny

4.1 Reason for Selection

- Lightweight transformer
- CPU-friendly
- Lower memory footprint
- Compatible with Hugging Face Trainer API

Maximum sequence length = 128.

5 Dataset Description

8 balanced genres:

- Romance
- Mystery Thriller Crime
- Young Adult
- History Biography
- Comics Graphic
- Fantasy Paranormal
- Children
- Poetry

Total evaluation samples = 1600.

6 Training Configuration

- Optimizer: AdamW
- Loss Function: CrossEntropyLoss
- Evaluation Strategy: Per epoch
- Metrics: Accuracy and Weighted F1-score

7 Local Evaluation Results

Metric	Value
Eval Loss	3.5703
Accuracy	0.0956
Weighted F1	0.0967

The model shows clear underfitting due to limited capacity and CPU-only training.

8 Docker Evaluation Results

Metric	Value
Eval Loss	3.6730
Accuracy	0.1181
Weighted F1	0.1122

9 Comparison: Local vs Docker

Metric	Local	Docker
Eval Loss	3.5703	3.6730
Accuracy	0.0956	0.1181
Weighted F1	0.0967	0.1122

Docker achieved slightly higher accuracy and F1-score. Minor differences may be due to seed or environment variations.

10 Model Deployment to Hugging Face Hub

Repository:

Akanksha8822/tiny_model_ak

10.1 Upload Commands

```
model.push_to_hub("Akanksha8822/tiny_model_ak")
tokenizer.push_to_hub("Akanksha8822/tiny_model_ak")
```

11 Docker Image Creation and Execution

11.1 Training Image

```
docker build -t ak -f Dockerfile.train .  
docker run -e HF_TOKEN="**" ak python /src/train.py
```

11.2 Evaluation Image

```
docker build -t ak1 -f Dockerfile.eval .  
docker run -e HF_TOKEN="**" ak1 python /src/evaluate.py
```

Model loading inside container:

```
AutoModelForSequenceClassification.from_pretrained(  
    "Akanksha8822/tiny_model_ak"  
)
```

11.3 GitHub Repository

The complete source code, Docker configuration, and training pipeline are available at:

https://github.com/AkankshaKapil/MLOPS_Akanksha_M25CSA033/tree/Assignment_3

11.4 Hugging Face Model Repository

The trained model is publicly hosted on Hugging Face Hub:

https://huggingface.co/Akanksha8822/tiny_model_ak

12 Challenges Faced

- CPU-only training increased runtime
- Low accuracy due to small model size
- Managing Hugging Face authentication

13 Limitations

- Low overall accuracy (11.8%)
- No hyperparameter tuning
- No learning rate scheduler

14 Future Improvements

- Use larger transformer models
- Increase training epochs
- Train using GPU

15 Conclusion

This project successfully demonstrates a complete end-to-end Machine Learning workflow through the following sequential steps:

- **Step 1: Notebook Development**
Initial experimentation and model development were performed using a Jupyter notebook.
- **Step 2: Modular Script Conversion**
The notebook was converted into structured Python modules (`data.py`, `train.py`, `evaluate.py`) to ensure maintainability and production readiness.
- **Step 3: Model Training**
A pre-trained transformer model (`prajjwal1/bert-tiny`) was fine-tuned using the Hugging Face Trainer API.
- **Step 4: Local Evaluation**
The trained model was evaluated locally using Accuracy and Weighted F1-score.
- **Step 5: Model Deployment**
The fine-tuned model and tokenizer were uploaded to Hugging Face Hub .
- **Step 6: Docker Containerization**
Separate Docker images were created for training and evaluation to ensure reproducibility.
- **Step 7: Docker-Based Re-evaluation**
The model was pulled from Hugging Face inside a Docker container and re-evaluated to validate deployment consistency.
- **Step 8: GitHub Publication**
The complete project, including source code and Docker configurations, was published to GitHub.

The final pipeline is reproducible, containerized, publicly deployable, and production-ready.